

Universal Agritech Marketplace Platform

Technical & Commercial Implementation Architecture for the Nepalese Agricultural Ecosystem

Prepared For: Stakeholders & Integration Partners

Date: June 2026

Prepared By: Nirajan Khatriwada, Project Director

Status: Production Design Proposal

1. EXECUTIVE SUMMARY & CORE PLATFORM VISION

The modernization of agricultural distribution within Nepal requires synchronous, transparent data flows between grassroots cultivators and commercial bulk buyers. This proposal establishes the architecture for a **Universal Agritech Marketplace Platform** designed specifically to link local producers and procurement entities.

The platform's business model mitigates the high fragmentation observed in local markets by acting as a centralized request ledger managed directly by company administration. Instead of navigating separate operational architectures that inflate overhead costs, the solution merges all front-end environments into a single software lifecycle. Utilizing **Expo Router with Platform Extensions**, the framework dynamically renders mobile apps optimized for native storefronts alongside a full-scale web-accessible command module.

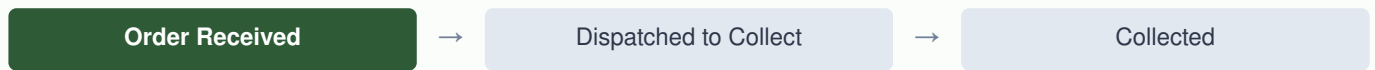
2. STAKEHOLDER PROFILES & FUNCTIONAL WORKFLOWS

The marketplace model relies on distinct behavioral interfaces matching user environments. To support wide accessibility, the user layer supports on-the-fly toggling between English and Nepali across all features.

2.1 Farmer / Seller Module (Mobile Interface)

Farmers require an immediate, lightweight workflow configured for out-of-field operations. Authenticating securely via their primary **Mobile Phone Number and Password**, they gain access to a tailored catalog:

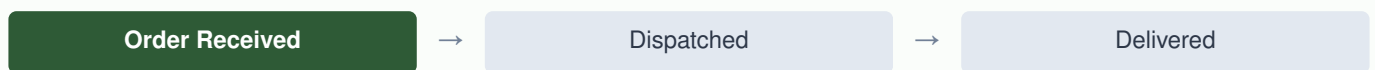
- **Produce Provisioning:** Cultivators select crops from a pool dynamically published by administration. Input forms enforce volumes strictly measured in Kilograms (KGs).
- **Logistic Point of Origin:** Profiles explicitly require a registered **Pick-up Location Address** alongside contact coordinates (Phone/Email) to guide internal freight logistics.
- **7-Day Bikram Sambat (BS) Ready Slider:** A specialized calendar component displays an immediate 7-day rolling window rendered in the local calendar scheme (e.g., Ashar 4 to Ashar 10), enabling users to select the precise date their yield will be ready to load.
- **Supply Status Lifecycle:** A transactional pipeline keeps users updated via a step-by-step progress tracking panel:



2.2 Wholesaler / Buyer Module (Mobile Interface)

Commercial buyers interact via a parallel mobile configuration designed to capture market demand. Registration tracks corporate entity identities using mobile authentication:

- **Procurement Sourcing:** Wholesalers select bulk vegetables or processing inventory (such as custom regional pickles), logging precise volume limits in KGs.
- **Drop-off Logistics:** Every profile includes a dedicated **Drop-off Location Address** mapping to the buyer's warehouse or retail site.
- **7-Day Delivery Scheduler:** Utilizing the integrated Bikram Sambat layout framework, the buyer chooses their preferred delivery date within a 7-day operational window.
- **Fulfillment Status Lifecycle:** Tracks progress down to final drop-off verification:



2.3 Central Administration (Desktop Web Module)

Accessed exclusively through a dedicated enterprise subdomain (e.g., `admin.yourcompany.com`), the admin center provides complete management control. It features an interactive grid designed for wide screens, allowing admins to oversee orders, verify profiles, adjust matching rules, and manage inventory settings.

3. PLATFORM TECHNICAL ARCHITECTURE

To combine quick development turnaround with solid security boundaries, the system follows a **Single Codebase, Platform-Isolating Framework** strategy using Expo's web rendering layer.

Strategic Architecture Isolation (.web.tsx Pattern): By explicitly suffixing administrative system modules with `.web.tsx` extensions, the underlying compiler blocks administrative assets from entering mobile production binaries. When running `eas build`, all admin code is stripped away, maintaining a small package footprint and secure code division.

The system directory maps out as follows within the source repository:

```
my-agritech-platform/
├─ app/
│  ├─ _layout.tsx # Global Auth Guards, Contextual Translation Engines
│  ├─ index.tsx # Universal Authentication Routing Director
│  ├─ (auth)/ # Shared Account Provisioning Components
│  │  ├─ login.tsx # Phone Number / Password Validator
│  │  └─ register.tsx # Multi-Role Location & Profiling Matrix
│  └─ (mobile)/ # Native App Bundle Layers (Farmer & Wholesaler)
```

```

| | └─ _layout.tsx # Tab Bar Navigators
| | └─ farmer.tsx # Supply Post Interface
| | └─ wholesaler.tsx # Procurement Order Entry
| └─ (admin)/ # Static Web Module (Admin Command Center)
| └─ index.web.tsx # Double Stream Match Matrix
| └─ inventory.web.tsx # Catalog CRUD & Asset Manager
| └─ error.tsx # Platform Access Guard (Mobile Deny Screen)
└─ app.json # Native Configuration File
└─ package.json # Dependency Manifesto

```

4. IMPLEMENTATION LOGIC & RELATIONAL DATA MODEL

Transactions connect transparently via alphanumeric order IDs generated during item creation. The admin layout renders active buyer and seller streams side-by-side, drawing parameters directly from a structured data schema.

Table Name	Core Schema Attributes	Relational Constraints & Types
users	id, phone, password_hash, email, role, primary_address	UUID Primary Key, Phone constraint (Unique), Role Enum (FARMER, BUYER, ADMIN)
catalog	id, crop_name, image_url, category, is_available	UUID Primary Key, Text naming convention, Boolean availability flag
orders	id, order_id, client_id, crop_id, weight_kg, target_date_bs, layer_type, status	UUID Primary Key, Unique AlphaNumeric Order string, Client Reference Foreign Key, Layer Enum (SUPPLY, DEMAND)
inventory_ledger	id, crop_id, delta_quantity, tracking_type, recorded_at	Foreign Key reference to Catalog, Numeric delta values, Enum tracking flags

4.1 Internal Stock Calculation Engine

To track real-time stock balances across the warehouse, the admin control center runs an internal ledger calculation function. The total stock for any catalog entry is computed as the sum of items received minus the total volume successfully delivered to wholesalers:

$$S_{\{available\}} = \sum Q_{\{receipts\}} - \sum Q_{\{delivered\}}$$

The admin interface allows staff to easily update these parameters. Using the *Stock Adding Option*, administrators can log newly received produce and immediately update stock levels across both mobile customer profiles.

5. DEPLOYMENT PIPELINE & INFRASTRUCTURE

Once verified within the local development environment, the deployment pipeline splits duties cleanly across target environments without duplicating code maintenance tasks.

Pipeline Target	Execution Command	Target Infrastructure	Security Boundaries
Native Mobile App Layer	<code>eas build --platform all</code>	Google Play Console / Apple App Store Connect	Excludes web files; handles security via client-side OAuth sessions.
Operations Web Layer	<code>npx expo export --platform web</code> <code>eas deploy</code>	EAS Global Edge Hosting Infrastructure	Enforces strict path checks, auto-blocking mobile view links.

Strategic Conclusion:

This universal architecture balances cross-platform performance with operational security. By choosing a single-codebase model with platform extensions, OpsAgora can deploy a highly tailored, localized product for Nepal's agritech sector while keeping development timelines and deployment costs low.